Sparse Goal Edits and Variable-Duration Options in World-Model Hierarchical RL

George Vasilache

Working draft — July 14, 2026

Abstract

We study whether the manager of a Director-style hierarchical agent built on DreamerV3 can act more sparsely — editing only part of a persistent goal per decision and choosing how long each goal is held — without losing control performance. On dense-reward cartpole the combined recipe reaches returns of ∼840 while editing fewer goal dimensions per step than the fixed-schedule baseline, and outperforms each of its components in isolation; on cheetah, replacing fixed penalty weights with Lagrangian constraint controllers roughly doubles the return. On sparse-reward tasks (hopper hop, acrobot swingup) every restricted variant fails to ever encounter reward, while the unrestricted baseline hierarchy learns both. Controlled comparisons isolate two candidate mechanisms — short goal durations and an unobservable, random per-goal horizon on the worker side — and we extend the agent with timed subgoals (a countdown-conditioned worker, following Gürtler et al. 2021) to test the latter. A factorial experiment across duration target, structural code regularization, and countdown conditioning is in progress; preliminary signal (single seed, 50–83% of budget) shows the countdown substantially rescues a collapsing dense-task variable-duration run but does not move either sparse task, and that masked edits alone (fixed schedule, no variable duration) are *independently* sufficient to remove hopper's reward too — so the two restrictions are each, on their own, enough to break sparse-task learning.

1 The base agent

The base agent is a reimplementation of Director (Hafner et al. 2022) on DreamerV3. All components below are standard; the paper is self-contained given this section.

World model. A recurrent state-space model (RSSM) is trained on replayed experience to predict observations, rewards, and episode continuation. It provides latent states $s_t = (\text{deter}_t, \text{stoch}_t)$, and both policies are trained entirely on *imagined* rollouts of this model (horizon H=16), not on real transitions.

Goal autoencoder. Goals are latent states compressed through a discrete autoencoder: an encoder maps a target latent to a code $z \in \{1, \dots, C\}^L$ of L categorical blocks (L=8, C=8 here), a decoder maps codes back to latent goals. The manager acts in code space; the worker receives the decoded latent goal g.

Manager. Every K steps (K=8 in Director) the manager policy $\pi^{\text{mgr}}(z \mid s)$ emits a goal code. It is trained by REINFORCE on imagined rollouts to maximize task (extrinsic) reward plus a novelty (exploration) reward with weight 0.1, each with its own critic. The per-decision reward is the *mean* of the per-step rewards over the goal's hold window, so the manager's return is invariant to how finely time is partitioned. An entropy constraint holds the manager's normalized policy entropy near a target of 0.5.

Worker. The worker policy $\pi^{\text{wkr}}(a \mid s, g)$ receives the state features and the decoded goal, and is trained on a dense per-step reward: the (max-)cosine similarity between the current latent and the goal. Its λ-return is cut at goal switches, so its value function $V(s, g)$ bootstraps only within the current goal's window. The worker *never* observes task reward. This division — manager selects goals for task reward, worker reaches goals — is what the rest of the paper perturbs.

2 Two manager restrictions

We ask whether manager decisions can be made *sparser* along two axes, with the long-term aims of interpretability (attributable, local goal edits), a controllable temporal-abstraction knob, and less non-stationarity in the worker's objective (most of the goal persists across decisions).

Masked goal edits. The manager maintains a persistent goal code Z_t and at each decision outputs a candidate code z_t together with a binary mask $m_t \in \{0, 1\}^L$ over the L blocks, updating

$$Z_t \;=\; m_t \odot z_t \;+\; (1 - m_t) \odot Z_{t-1}, \tag{1}$$

so only the selected blocks change. The mask is a learned policy output; its sparsity is controlled by constraints (Sec. 3), not hard-coded.

Variable durations. Alongside the goal, the manager outputs a hold duration $d_t \in \{1, \ldots, 16\}$ from a categorical head; the next decision occurs after exactly d_t steps (the worker is never interrupted early).

The combined cost measure is the average number of goal blocks edited per environment step,

$$\text{blk/step} \;=\; \frac{\mathbb{E}[\|m\|_1]}{\mathbb{E}[d]} \;=\; \frac{\bar{m}\,L}{\bar{K}}, \tag{2}$$

with $\bar{m}$ the mean edit fraction and $\bar{K}$ the realized mean duration. Director corresponds to $\bar{m}$=1, $\bar{K}$=8: blk/step = 1.

3 Learning machinery

3.1 Constraint controllers

Direct fixed penalties on edit fraction or duration proved brittle and task-specific. The current recipe controls both with dual-ascent Lagrangian controllers of a common form: for a statistic x with target τ, a multiplier λ on the term λx in the manager loss is updated

$$\lambda \leftarrow \text{clip}\big(\lambda \cdot \exp(\eta\,(x - \tau)),\ \lambda_{\min},\ \lambda_{\max}\big). \tag{3}$$

Three controllers are active in the combined recipe: (i) **edit rate**: mean mask probability $\to 0.3$; (ii) **per-block mask entropy**: $\to 0.5$, preventing collapse onto always/never-edited blocks; (iii) **duration**: the switch-weighted deviation $|\mathbb{E}[d] - \tau_d| \to$ tolerance 0.1 (with $\tau_d \in \{4, 8\}$); regulating the deviation keeps the controller symmetric, and it relaxes to $\lambda \to 0$ once within tolerance.

3.2 Structural code consistency

A structural loss with weight w_s (200 unless stated) ties code-space distances to latent-state distances so that small code edits correspond to nearby goals:

$$\mathcal{L}_{\text{struct}} = \big\|\text{dist}(z_i, z_j) - \text{dist}(s_i, s_j)\big\|^2, \qquad \text{realized corr.} \approx 0.92\text{–}0.98. \tag{4}$$

This acts on the goal autoencoder independently of masking. It has two faces: it *stabilizes* what codes mean while the autoencoder trains on an expanding state distribution, and it *localizes* proposals (small code moves = small goal moves), which may suppress exploration on sparse tasks. Separating these two effects is one goal of the current experiments. A first adaptive variant regulated w_s (Eq. 3) toward a small, two-sided setpoint (0.01) on the raw structural loss; diagnosing why it underperformed a fixed w_s=200 on cheetah (e198 vs. e174, single seed) showed the raw loss clears that setpoint almost immediately, collapsing the multiplier from its init of 200 to ∼20–30 within the first 7% of training and leaving the diagnostic correlation (Eq. 4) 3–6 points below the fixed-weight run's for the remainder. Retargeting onto the correlation directly (0.97) was considered but rejected after checking the fixed-weight runs' own trajectories: at BIG scale the correlation never reaches 0.97 even under w_s=200 held constant for all 4M steps (e171 plateaus at 0.90–0.92, e175 at 0.94), so a 0.97 floor would simply ratchet the multiplier to its ceiling and pin it there for the whole run. The setpoint instead stays on the raw loss, lowered to 0.005 (below every fixed-weight run's floor at either scale) to 0.005 (below every fixed-weight run's floor at either scale), keeping the symmetric, two-sided form of Eq. (3) (grows above target, shrinks below). The controller implementation also gained an optional one-sided mode — grows on violation but never relaxes once the target clears — motivated by the same fixed-weight runs showing the loss can drift back above a setpoint later in training with no relaxation mechanism at play at all (e171: 0.0068 at 445k steps to 0.0183 at 3.1M), which would let a symmetric controller shrink early and then lag on re-detecting the later violation; it is available but not the default while the standard dual-ascent form is evaluated first. Results under this variant are pending.

3.3 Timed subgoals: countdown-conditioned worker

In the base agent the worker observes only (s, g): it cannot see how long it has to reach the goal, and under variable durations its value target has a *random, unobservable* horizon (1–16 steps, mean τ_d) — variance injected directly into its critic, on top of shorter reach-times at τ_d=4. Empirically, worker goal-reward is lowest exactly where variable-duration runs collapse. Following HiTS (Gürtler et al., NeurIPS 2021), which

shows that a lower level pursuing *timed* subgoals $(g, \Delta t)$ restores a stationary learning problem, we append the **countdown** — steps left on the current goal including the current one, normalized to $[-1, 1]$ by the duration budget — to the worker's policy and value inputs (`worker_timed_goals`). Our variable-duration manager already implements the manager half of timed subgoals (durations fixed in advance, no early interruption); this supplies the missing worker half. The HiTS machinery that relies on off-policy replay relabeling (hindsight action relabeling, HER with time relabeling, deadline-only sparse rewards) does not transfer to imagination training and is not used; the worker keeps its dense cosine reward, so the experiment isolates *horizon observability*. The formal definition of the countdown and its place in the worker's losses is given in Sec. 3.4.

3.4 The complete objective

All losses are combined as a scale-weighted sum over independent keys, $\mathcal{L} = \sum_k s_k \mathcal{L}_k$; the structural invariant carried through this project (Sec. 5, finding 2) is that *no constraint shares a key — and hence a scale or gradient clip — with a policy-gradient term.* The world-model losses (reconstruction, reward, continuation, dynamics/representation KL) are standard DreamerV3 and untouched. The hierarchy adds:

Goal autoencoder (key `goal_autoencoder`): reconstruction of the latent target, a KL to a uniform code prior, and the structural term of Eq. (4) — weighted by fixed w_s or by the adaptive multiplier λ_s of the pilot variant:

$$\mathcal{L}_{\text{AE}} = \big\|\text{dec}(\text{enc}(s)) - s\big\|^2 + \beta\,\text{KL}\big(\text{enc}(s) \,\|\, \text{uniform}\big) + \lambda_s\,\mathcal{L}_{\text{struct}}, \qquad \beta = 0.25. \tag{5}$$

Manager actor (key `mgr_policy`): REINFORCE over decision steps on a normalized two-critic advantage, plus entropy *constraints* on both output heads — these are also adaptive multipliers, holding each head's normalized entropy near 0.5 rather than adding a fixed bonus:

$$\mathcal{L}_{\text{mgr}} = -\,\mathbb{E}\Big[\log \pi^{\text{mgr}}(z_t, m_t, d_t \mid \cdot)\,\hat{A}_t\Big] + \sum_{h \in \{\text{skill},\,\text{dur}\}} \lambda_h\big(\tau_H - \bar{H}_h\big), \qquad \hat{A}_t = \text{norm}\big(A_t^{\text{extr}} + 0.1\,A_t^{\text{expl}}\big), \tag{6}$$

with separate critics (and replay-value twins) for the extrinsic and exploration returns. The sparsity and duration constraints sit in their own keys exactly as in Eq. (11): `mask_sparsity` carries $\lambda_m(\bar{p} - \tau_m) + \lambda_H(\tau_H - \bar{H}_{\text{block}})$ and `goal_duration_prior` carries $\lambda_d(\mathbb{E}[d] - \tau_d)^2$.

Worker actor-critic (keys `wkr_policy`, `wkr_goal_value`, + replay twin), where the countdown enters. Let n_t be the number of steps left on the current goal *including* the current one ($n_t = d$ at the decision step, then decrementing), and

$$c_t \;=\; \frac{2\,\min(n_t,\, d_{\max})}{d_{\max}} - 1 \;\in\; [-1, 1], \qquad d_{\max} = 16 \text{ (var-}K\text{) / } K \text{ (fixed)}. \tag{7}$$

With `worker_timed_goals` the worker's policy *and* critic are conditioned on it: $\pi^{\text{wkr}}(a \mid \text{deter}, \text{stoch}, g, c_t)$ and $V^{\text{wkr}}(\text{deter}, \text{stoch}, g, c_t)$, identically at training and acting time (imagination rollout, imagination losses, replay value, and the real environment step all thread the same counter). The worker reward and return are

$$r_t^{\text{wkr}} = \cos_{\max}\big(g_t,\, \text{deter}_t\big), \qquad G_t = \lambda\text{-return}\big(r^{\text{wkr}},\, V^{\text{wkr}}\big) \text{ reset at goal switches}, \tag{8}$$

and the losses are standard REINFORCE with an entropy bonus $(-\log \pi \cdot \text{sg}(\hat{A}^{\text{wkr}}) - \beta_H H)$ and the critic NLL against G_t. The reset in Eq. (8) makes each hold window its own value segment (Director's `split_traj`); the point of Eq. (7) is that this truncation is only well-posed if the worker can *see* where the segment ends. Without c_t, the same (state, g) pair carries return targets computed over horizons of 1–16 steps drawn from the manager's duration head — irreducible variance injected into the critic. With c_t, $V(\cdot, c_t)$ is a deterministic function of an observable horizon, and the policy can calibrate effort to the time budget (attempt near goals when c_t is low, commit to far goals when high). No term of any loss changes; only the two worker function approximators gain an input.

3.5 The combined recipe

"Combined recipe" below always means: Director base (Sec. 1) + masked edits (Eq. 1) + variable durations + the three controllers of Sec. 3 (edit-rate 0.3, mask entropy 0.5, duration τ_d with tolerance 0.1) + structural loss w_s=200. "Variable-K only" means variable durations with a weak fixed duration prior (weight 0.01) instead of the Lagrangian, no masking, w_s=0. "Mask only" means masked edits at fixed K=8, w_s=200.

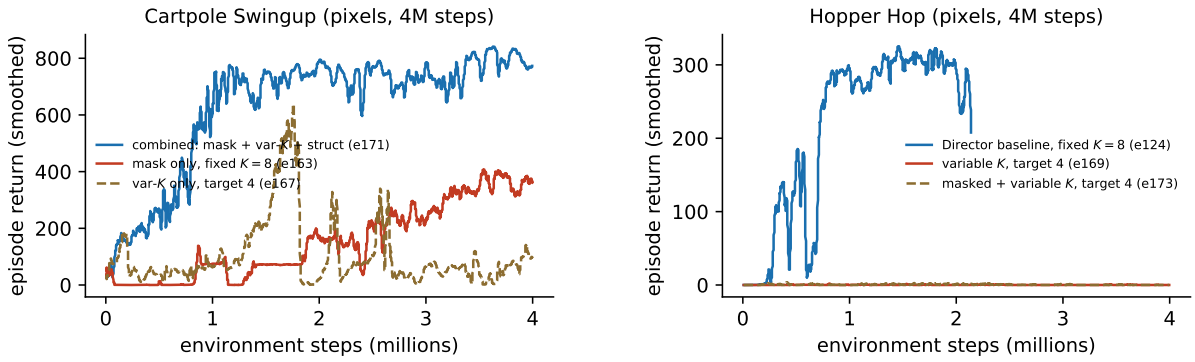

Figure 1: Left: on cartpole the combined recipe outperforms either component alone. Right: on hopper the Director baseline (fixed K=8, full goal replacement) learns, while the variable-duration variant with target 4 — differing *only* in the variable-duration package — never encounters reward; adding masking does not change this.

4 Experimental setup

Tasks are DeepMind Control from pixels (+proprioception): cartpole swingup and cheetah run (dense reward), hopper hop and acrobot swingup (sparse/impulsive reward). Two scales (Table 1); runs are 4M environment steps, single seed (replication is acknowledged, pending, and flagged wherever a claim rests on one run). Experiments carry global numbers (`e`n) recorded in the project log, where hypotheses, quantitative expectations, and outcome branches are *pre-registered before results*; scores are episode returns (max 1000), reported as last-10-episode means and best trailing-10 window ("peak").

Scale	Image	Model	Batch	Hardware
small	32^2	6M params	16×32	1×V100
BIG	64^2	Director-matched (deter 1024, 512-wide MLPs)	16×64	1×A100-80GB

Table 1: Run scales. BIG matches the published Director architecture; all cross-recipe comparisons are within-scale.

5 Results to date

Task	Recipe (BIG unless noted)	last10	peak	Run
cartpole	combined recipe (τ_d=4)	776	844	e171 (repro of e157: 833/
cartpole	mask only, fixed K=8	360	419	e163
cartpole	variable-K only, τ_d=4, w_s=0	97	658	e167 (rose, then collapsed
cartpole	variable-K + struct, τ_d=8 (small)	725	758	e46
cheetah	combined, Lagrangian controllers	465	508	e175
cheetah	combined, fixed penalty weights	253	–	e142
hopper	mask only, fixed K=8 (no variable duration)	0	4.5	e164/e165
hopper	*any* masked / variable-K variant	0	∼6	20+ runs
hopper	Director baseline, fixed K=8	180	326	e124
acrobot	combined recipe	2	26	e176/e177
acrobot	Director baseline	learning (3.3M: 197, peak 295, still rising)		e180, in progress

Table 2: Key results. Single seeds; returns are episode scores (max 1000).

1. The components interact; neither suffices alone. On cartpole, masked edits at fixed K=8 and variable durations without masking/struct both fall far short of the combined recipe (Table 2, Fig. 1 left). This matches program history: masked fixed-K=8 variants collapsed or plateaued repeatedly, and plain variable-K has only worked with both a duration prior and the structural loss (724, e46). The combined recipe achieves blk/step ≈ 0.6–1.1 at returns of 776–844 (Director: blk/step = 1).

2. Two-sided targets, then Lagrangian enforcement — the staged rescue of large-scale performance. Director-scale performance was recovered in two steps, each changing only how the manager's edit rate and duration are regularized. The three formulations, in the order they were tried:

Stage 0 — priced one-sided costs (underperforms). The first BIG runs added fixed costs to the manager objective and left duration free,

$$\mathcal{L}^{(0)}_{\text{mgr}} \;=\; \mathcal{L}_{\text{RL}} \;+\; c_{\text{sw}}\,\mathbf{1}[\text{switch}_t] \;+\; c_{\text{edit}}\,\|m_t\|_1. \tag{9}$$

Both penalty terms are *one-sided*: their gradient pushes editing and switching down with constant sign, and the only opposing force is the task advantage through a sampled, stop-gradiented mask — too weak to balance. The system therefore has no interior fixed point and runs to corners: the mask freezes ($\|m\|\to 0$, goal never updated, return $\to 0$) or saturates, and the duration head drifts to near-maximal holds ($\bar{K}\approx 15$–21; the manager stops re-editing and goals go stale). Outcome: cartpole 528, cheetah 346, below the 435 baseline.

Stage 1 — fixed two-sided targets (recovers cartpole only). Reinstating explicit setpoints,

$$\mathcal{L}^{(1)}_{\text{mgr}} \;=\; \mathcal{L}_{\text{RL}} \;+\; w_d\left(\mathbb{E}[d]-\tau_d\right)^2 \;+\; \lambda_m\left(\bar{p}-\tau_m\right), \tag{10}$$

restores an interior equilibrium (the quadratic pulls from both sides). The two sides are structurally *asymmetric* here: the mask term $\lambda_m(\bar{p}-\tau_m)$ was already dual-ascent *and* already lived in its own independently-scaled loss key, whereas the duration quadratic is folded directly into the manager's policy loss, sharing its scale and gradient clip. That asymmetry is where both remaining problems live. First, w_d is scale-fragile: early in training $(\mathbb{E}[d]-\tau_d)^2$ can be $\sim$64, so w_d=0.1 contributes an $O(1+)$ loss that dominates the percentile-normalized REINFORCE term it shares a scale with — the manager optimizes the prior instead of the task ("magnitude domination"); only $w_d \le 0.03$ coexists. Second, the workable w_d is task-specific and does not transfer. The mask side, already separated, exhibits neither problem. Outcome: cartpole 709, cheetah 253.

Stage 2 — same targets, symmetric dual-ascent enforcement (recovers both). The fix makes the duration side structurally identical to the mask side: the fixed weight becomes a multiplier updated by Eq. (3), and the duration term moves *out* of the policy loss into its own independently-scaled loss key — mirroring where the mask term already was — so no constraint shares the policy gradient's scale or clip. A per-block entropy floor is added on the mask:

$$\mathcal{L}^{(2)}_{\text{mgr}} \;=\; \mathcal{L}_{\text{RL}} \;+\; \underbrace{\lambda_d\left(\mathbb{E}[d]-\tau_d\right)^2}_{\text{key: goal\_duration\_prior}} \;+\; \underbrace{\lambda_m\left(\bar{p}-\tau_m\right) \;+\; \lambda_H\left(\tau_H-\bar{H}_{\text{block}}\right)}_{\text{key: mask\_sparsity}}, \tag{11}$$

where λ_d is stepped on the switch-weighted deviation $|\bar{\mathbb{E}}[d]-\tau_d|$ against a tolerance of 0.1 (symmetric, so it relaxes from either side) while the penalty it scales is the per-decision squared deviation; $\bar{p}$ is the mean edit probability; and $\bar{H}_{\text{block}}$ the per-block mask entropy whose floor τ_H prevents collapse onto always/never-edited blocks. Each λ grows only while its constraint is violated and decays otherwise, so enforcement strength is learned per task while the *setpoints stay fixed*. Mechanism metrics confirm the design: duration std $\sim$20$\times$ tighter than Stage 1 (0.0012 vs 0.024), mask pinned at 0.31–0.35, and at convergence $\lambda_d \to 0$ — the constraint holds with no active pressure. Outcome: cartpole 833, cheetah 465–508.

The converse design — fully adaptive targets with no fixed setpoints — kept statistics interior but learned nothing off cartpole. The transferable recipe is therefore a *fixed target with an adaptive multiplier*: the target supplies the missing gradient direction (Stage 0's failure), the multiplier supplies the task-appropriate magnitude (Stage 1's failure).

3. Sparse-reward tasks fail under every restricted variant. On hopper, every masked and/or variable-duration configuration tested (16+ runs across recipes, scales, controllers) shows manager extrinsic reward $\approx 10^{-4}$ throughout: reward is never encountered, so nothing downstream can learn. The Director baseline finds reward within 0.2–0.4M steps. The same baseline is now visibly learning acrobot while the combined recipe stays at zero there — the failures are caused by our restrictions, not by task difficulty for the base hierarchy.

4. Two independently sufficient restrictions kill hopper — variable duration and masking alone, separately. Resolved-configuration diffs show the variable-K-only hopper run and the Director baseline differ *only* in the variable-duration package (duration head, weak prior toward τ_d=4) — and that

difference separates 180 (peak 326) from exactly 0 (Fig. 1 right). Raising the target to 8 alone lifts the extrinsic reward rate from $\sim 10^{-4}$ to a noisy 10^{-3}–10^{-2} band that never sustains above the alive threshold through 3.2M steps — real but weak, and not rising further; duration-return coupling via sum aggregation did not help either (dead at 0.6M, cancelled). Separately, mask only at a *fixed* K=8 — the same schedule as the alive Director baseline, with masking as the only change — is also dead (0/4.5, Table 2). Since neither restriction needs the other to break hopper, hold length is not the single root cause we initially framed it as: masking and variable duration are each, independently, sufficient to remove sparse-task reward, and the surviving common factor is the unrestricted Director hierarchy itself. This reframes the search: rather than attributing the failure to one axis, the open question is what *any* deviation from whole-code, fixed-schedule goal replacement removes that sparse exploration depends on.

5. The collapse anatomy points at code drift and worker unreliability. In the variable-K-only cartpole run the manager's advantage grew to 4× its healthy level, then crashed to ≈ 0 permanently — at the same moment the worker's goal-reaching reward dropped to its run-low (0.14–0.21 vs. 0.32–0.46 in stable runs) and the goal autoencoder's reconstruction error kept climbing with *no structural anchor* (w_s=0). Policy entropies stayed pinned at their targets throughout — the collapse is a manager–worker–autoencoder coordination failure, not an entropy effect. Two repairs are under test: the structural loss as a code stabilizer (Sec. 3.2), and making the worker's horizon observable (Sec. 3.3).

6 Experiments in progress

All 4M steps; "alive" on sparse tasks = extrinsic reward rate $> 10^{-2}$ by 1M steps (baseline reference). Cells cancelled at 0.6M for being flat while their comparators learned: sum aggregation (hopper), the combined recipe with τ_d=8 (hopper and acrobot), and 10× manager exploration (hopper).

Run	Configuration	Task	Scale	Question
e178	var-K τ_d=8, w_s=0	hopper	BIG	hold length alone (pulse: 3×10^{-3} @0.6M)
e179	var-K τ_d=8, w_s=0	cartpole	BIG	is struct needed at τ_d=8? (stability)
e180	Director baseline	acrobot	BIG	task-difficulty control (learning)
e185	e178 + countdown	hopper	BIG	horizon observability on the pulse
e186	combined + countdown	cartpole	small	countdown effect inside best recipe
e187	var-K τ_d=4 + countdown	cartpole	small	sharpest worker-confusion test
e188	mask fixed-K=8 + countdown	cartpole	small	phase observability at fixed K
e189	var-K τ_d=8 + struct 200	hopper	BIG	struct: stabilizer or locality killer?
e190	Director baseline	cartpole	BIG	missing Director-matched control
e191	Director baseline	cheetah	BIG	control redo (prior run stopped at 2.6M,
e192	var-K τ_d=8 + countdown	acrobot	BIG	best-guess sparse recipe vs. e180
e193	var-K τ_d=4 + struct 200	cartpole	small	completes duration×struct 2×2
e194	var-K τ_d=8, w_s=0	cartpole	small	completes 2×2
e195	var-K τ_d=4 + adaptive struct	cartpole	small	struct-as-Lagrangian pilot
e196–e199	full stack: combined + countdown + adaptive struct	4 tasks	small	does the all-adaptive stack cost anything

Table 3: Hypothesis matrix. The small-cartpole 2×2 (e46/e166/e193/e194) disentangles duration target from struct; e178/185/189 plus a later struct×countdown cell factorize the hopper rescue; e190/e191 provide the Director-matched dense-task controls. A cartpole run of the combined recipe with w_s=0 (short-queue) probes struct within the full recipe.

Readout logic (pre-registered): if the countdown cells (e185, e187) lift their counterparts, the worker's unobservable horizon is a confirmed bottleneck and timed subgoals become part of the recipe. If struct-on-hopper (e189) kills the target-8 pulse, struct's locality face dominates on sparse tasks and it should be scheduled or gated, not constant; if it strengthens the pulse, struct is primarily a code stabilizer and belongs in every variable-K run, with the adaptive version (e195) replacing the hand-tuned weight.

Preliminary readout (2026-07-14, single seed, 50–83% of a 4M-step budget — all cells still training). Countdown's effect is split cleanly by task type, not uniform: on cartpole, e187 (plain var-K τ_d=4 + countdown) reaches 750/757 against e166's 130/192 on the byte-identical config without it — the "lift" branch firing decisively, confirming worker confusion about *when* the goal changes is a real, primary cost of variable duration on dense tasks. On hopper, e185 stays statistically level with its non-countdown counterpart e178 (both in the collapse-zone on worker reliability, both at the reward-rate floor) — horizon

observability is not hopper's binding constraint, so countdown is not a universal sparse-task fix. Struct shows both of its predicted faces at once, on different cells: e189 (struct 200 added to the τ_d=8 hopper pulse) makes the already-weak pulse *weaker*, firing the "locality face dominates, gate it on sparse tasks" branch; meanwhile e160 (struct removed from the full combined recipe on dense cartpole) collapses from a working peak of 138 down to 22 with manager advantage flatlining at zero, extending struct's stabilizer role from plain variable-K (already established) to the masked combined recipe as well. At τ_d=8 specifically, struct turns out to be dispensable on dense tasks, but only at the larger scale: e179 (BIG, no struct) is stable at 740/755, while the identical recipe at small scale (e194) stays stuck near 150 — struct's dispensability at longer holds looks scale-dependent, not general. Finally, acrobot's best-guess sparse recipe (e192) stays at the dead floor while its own Director-baseline control (e180) climbs steadily over the same window, splitting acrobot from hopper as a separate failure mode rather than a shared one.

7 Outlook

On dense tasks the combined recipe matches strong fixed-schedule baselines at comparable or lower edit rates, and constraint controllers remove per-task penalty tuning. The open problem is sparse-reward tasks, where the baseline hierarchy learns and every restricted variant so far does not. Finding 4 now narrows that gap to a sharper question than the factorial was originally designed around: hold length, masking, struct, and worker horizon observability have each been tested in isolation or combination, and every cell that changes anything about the base hierarchy's whole-code, fixed-schedule goal replacement stays dead on hopper, while acrobot appears to fail for a distinct reason (alive under the unrestricted baseline, dead under every restricted recipe tried so far including the countdown-augmented one). No cell in the current factorial has yet produced an alive sparse-task run. Per the project's standing escalation order, the next interventions if the remaining cells (struct×countdown, the full adaptive stack) also fail are code-level rather than hyperparameter-level: mixing task reward into the worker's objective, then periodic non-local goal proposals (mask-free decisions every Nth switch) to let the manager reach distant goals that gradual masked edits cannot. Whichever combination first produces an alive sparse-task run defines the revised recipe, to be re-evaluated on all four tasks with seed replication before any claims are finalized.